EXERCISE 1 : Evaluate the conditions

```
int x = 8, y = 3, z = 8;
 bool ready = false;
```

Complete the table without running a program. Write both the Boolean result and its numeric value (0 or 1)

	Expression	true / false	0 / 1
a	x > y	True	1
b	x != z	False	0
c	(y >= 1) && (y <= 5)	True	1
d	(x < y) || (z == 8)	True	1
e	!ready	True	1
f	(x < y) || (z == 8) && (y > 5)	False	0

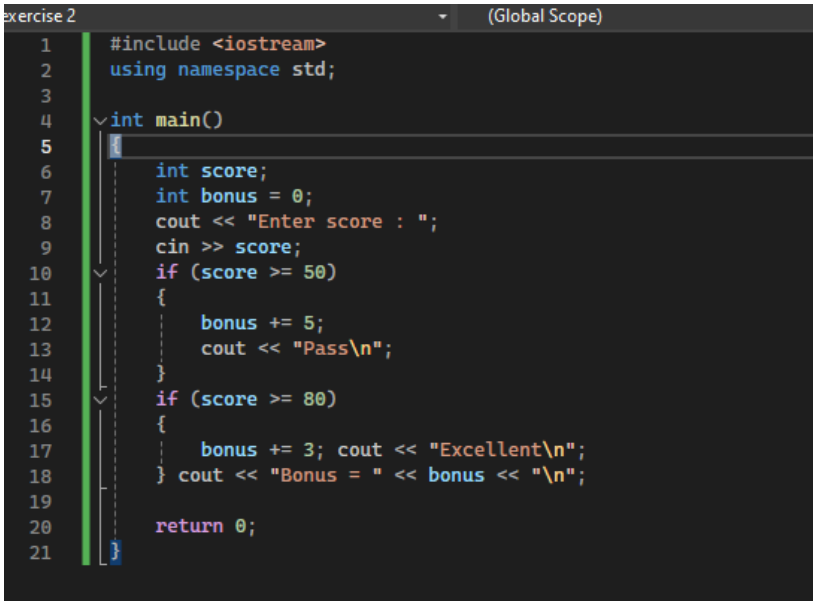
(g) Explain which logical operator is evaluated first in (f).

Operator && (logical AND) dinilai terlebih dahulu berbanding || (logical OR)

EXERCISE 2 : Trace independent if statements

Assume the following code is inside main().

```
int score;
int bonus = 0;
 cin >> score;
if (score >= 50)
{ bonus += 5;
 cout << "Pass\n";
}
if (score >= 80)
{ bonus += 3; cout << "Excellent\n";
} cout << "Bonus = " << bonus << "\n";
```



(a) Write the exact output for each input: 49, 50, 79, and 80.

49 = 0

50=5

79 =5

80 =8

(b) For an input of 80, how many if bodies execute? Explain why.

= Yes , both if statements are independent (they stand alone, not else if). Because the number 80 satisfies both conditions (80 >= 50 evaluates to true and 80 >= 80 also evaluates to true), both code blocks inside the if statements are executed sequentially.

(c) Does the final cout statement execute when the input is 49? Explain

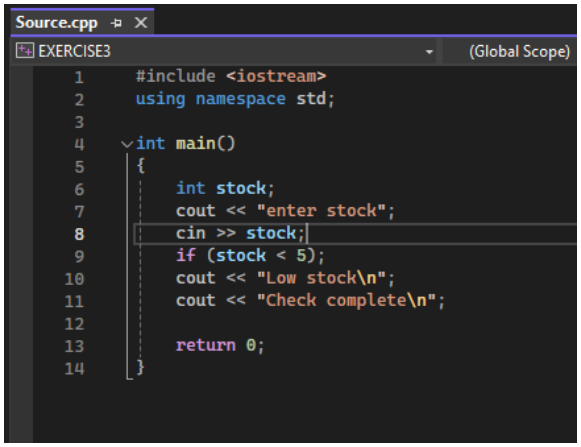
= Yes, the final cout statement is still executed. The statement cout << "Bonus = " << bonus << "\n"; is located outside of all if blocks. This statement is part of the program's normal execution flow, so it will always be executed regardless of the input value entered.

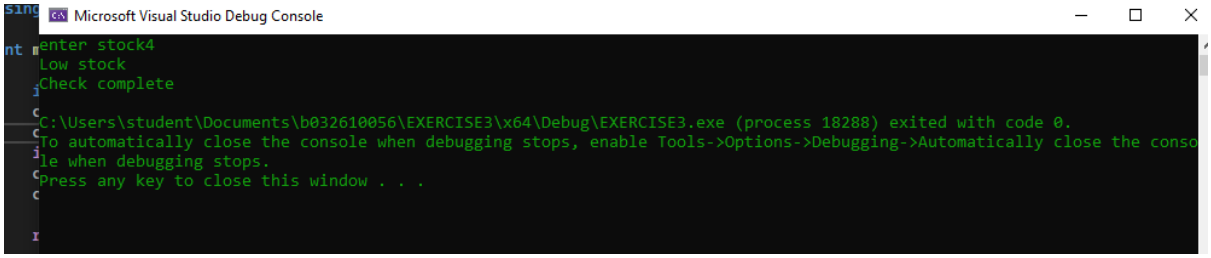
EXERCISE 3 : Find and fix the logic errors

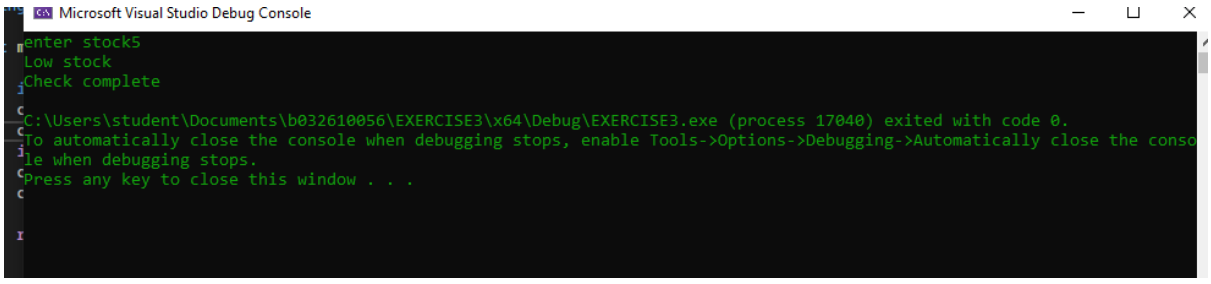
Each snippet is inside main(). For each part, explain the error and rewrite the affected statements so they match the intended behaviour.

(a) Low-stock warning. Display Low stock only when stock is less than 5. Always display Check complete. Test with 4 and 5.

```
int stock; cin >> stock;
 if (stock < 5);
cout << "Low stock\n";
cout << "Check complete\n";
```







(b) PIN check. Only when the PIN equals 2468, display Access granted and set attempts to 0. For any other PIN, attempts must remain 3. Test both cases.

```
int pin, attempts = 3;
cin >> pin;
 if (pin = 2468) cout << "Access granted\n";
 attempts = 0;
cout << "Attempts left = " << attempts << "\n";
```

SoalanB.cpp

soalanB

```cpp
#include <iostream>
using namespace std;

int main()
{
    int pin, attempts = 3;

    cout << "enter pin: ";
    cin >> pin;

    if (pin == 2468)
    {
        cout << "Access granted\n";
        attempts = 0;
    }

    cout << "Attempts left = " << attempts << "\n";

    return 0;
}
```

Microsoft Visual Studio Debug Console

```
enter pin: 2468
Access granted
Attempts left = 0

C:\Users\User\Downloads\soalanB\x64\Debug\soalanB.exe (process 9568) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .
```

Microsoft Visual Studio Debug Console

```
enter pin: 1234
Attempts left = 3

C:\Users\User\Downloads\soalanB\x64\Debug\soalanB.exe (process 12784) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .
```

(c) Valid mark. Display Valid mark only for marks from 0 to 100, inclusive. Test with -1, 0, 100, and 101

```
int mark;
cin >> mark;
if (0 <= mark <= 100)
 cout << "Valid mark\n";
```

SoalanC.cpp

soalanC (Global

```
#include <iostream>
using namespace std;

int main()
{
    int mark;

    cout << "enter mark: ";
    cin >> mark;

    if (mark >= 0 && mark <= 100)
    {
        cout << "Valid mark\n";
    }

    return 0;
}
```

Microsoft Visual Studio Debug Console

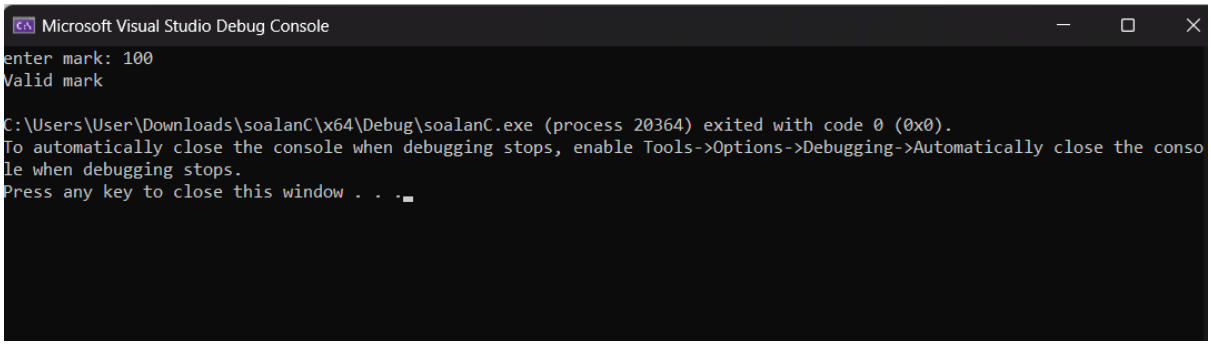
```
enter mark: -1

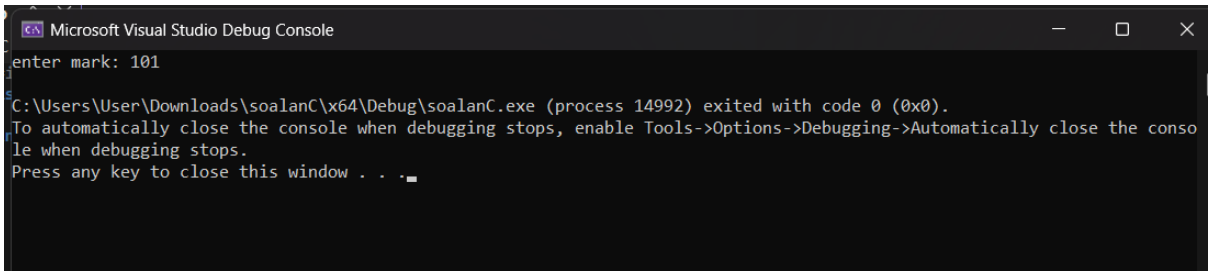
C:\Users\User\Downloads\soalanC\x64\Debug\soalanC.exe (process 9380) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .
```

Microsoft Visual Studio Debug Console

```
enter mark: 0
Valid mark

C:\Users\User\Downloads\soalanC\x64\Debug\soalanC.exe (process 23272) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .
```





EXERCISE 4: Campus shop discount

A campus shop gives a 10% discount when a purchase is RM100 or more. For a smaller purchase, the discount is RM0.00. Assume the purchase amount is a valid non-negative number.

(a) Prepare pseudocode or a flowchart for the solution.

Answer :

1. Start
2. Read purchase
3. Set discount = 0
4. If purchase >= 100 then
5. Discount = purchase * 0.10
6. End if
7. Payable = purchase - discount
8. Display discount
9. Display payable
10. End

(b) Write a C++ program that reads the purchase amount, initialises the discount to zero, and uses one if statement to calculate any discount.

```
EXERCISE 4 B                                                    (Global
 1  #include <iostream>
 2  using namespace std;
 3
 4  int main()
 5  {
 6      double purchase;
 7      double discount = 0.0;
 8
 9      cout << "enter purchase amount: RM";
10      cin >> purchase;
11
12      if (purchase >= 100.0)
```

```
Microsoft Visual Studio Debug Console
enter purchase amount: RM150
Discount = RM 15
Amount payable = RM135

C:\Users\User\Downloads\EXERCISE 4 B\x64\Debug\EXERCISE 4 B.exe (process 13264) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .
```

(c) Calculate and display the discount and final amount payable to two decimal places. Use for money formatting

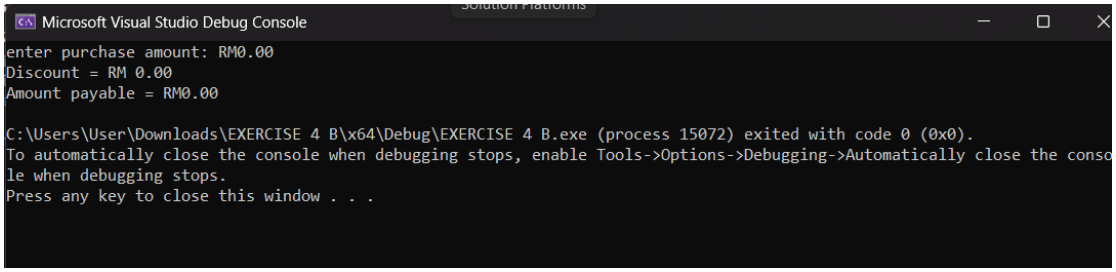
Sample input	Sample output (after the input)
150.00	Discount = RM 15.00 Amount payable = RM135.00

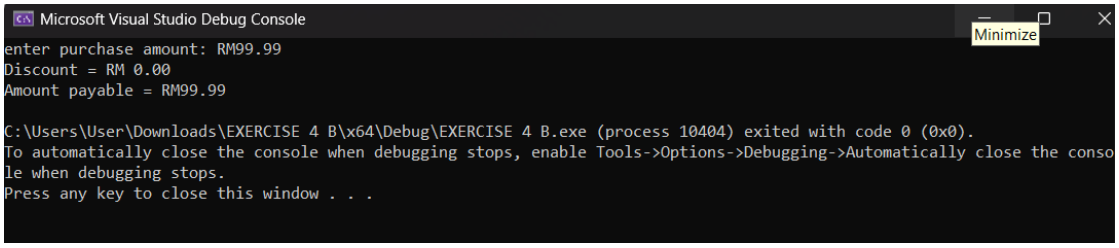
```
EXERCISE 4 B                                                    (Global Scope)
 1  #include <iostream>
 2  #include <iomanip>
 3  using namespace std;
 4
 5  int main()
 6  {
 7      double purchase;
 8      double discount = 0.0;
 9
10      cout << "enter purchase amount: RM";
11      cin >> purchase;
12
13      if (purchase >= 100.0)
14      {
15          discount = purchase * 0.10;
16      }
17
18      double payable = purchase - discount;
19
20      cout << fixed << setprecision(2);
21      cout << "Discount = RM " << discount << "\n";
22      cout << "Amount payable = RM" << payable << "\n";
23
24      return 0;
25  }
26
```

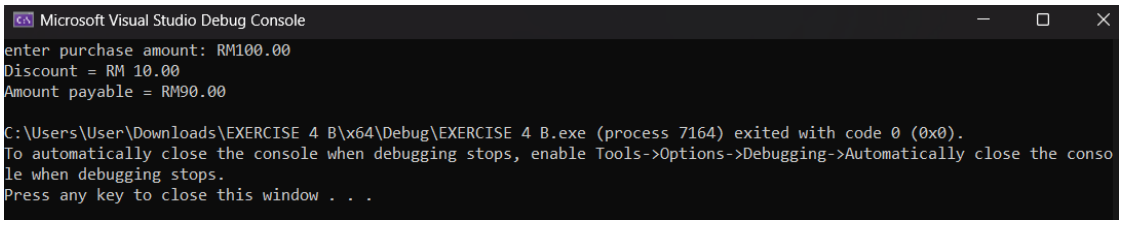
```
Microsoft Visual Studio Debug Console
enter purchase amount: RM150
Discount = RM 15.00
Amount payable = RM135.00

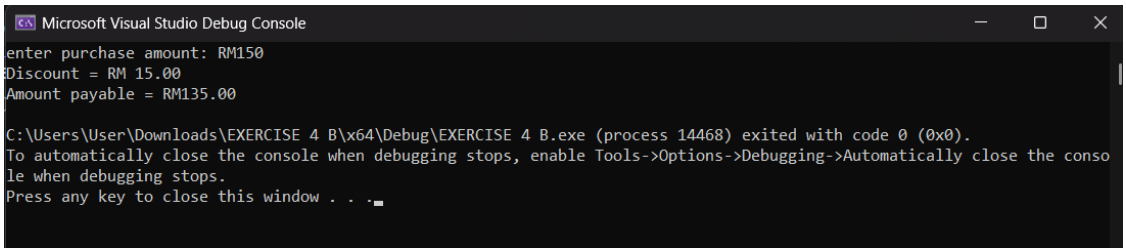
C:\Users\User\Downloads\EXERCISE 4 B\x64\Debug\EXERCISE 4 B.exe (process 19712) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .
```

(d) Test your program with RM0.00, RM99.99, RM100.00, and RM150.00. Record the expected and actual results.









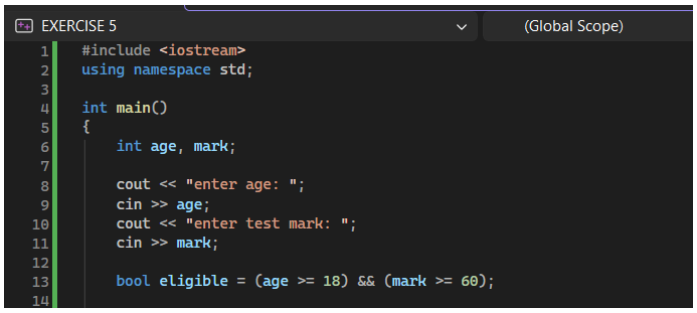
EXERCISE 5 : Workshop eligibility checker

A student can join a C++ workshop when both conditions are met: the student is at least 18 years old and has a prerequisite test mark of at least 60. Assume the age is non-negative and the mark is an integer from 0 to 100.

(a) Prepare pseudocode or a flowchart for the solution.

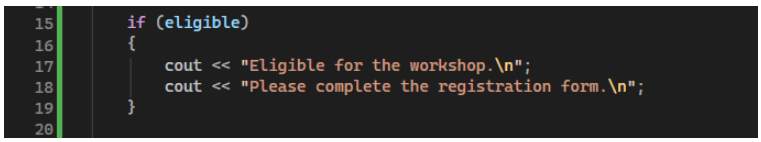
1. START
2. READ age, mark
3. eligible = (age >= 18) AND (mark >= 60)
4. IF eligible THEN
5. DISPLAY "Eligible for the workshop."
6. DISPLAY "Please complete the registration form."
7. END IF
8. IF NOT eligible THEN
9. DISPLAY "Not eligible for the workshop."
10. END IF
11. END

(b) Read the student's age and test mark. Declare a bool variable named eligible and assign it the result of a logical expression.



(c) Use if (eligible) to display both lines below in one body. Use braces.

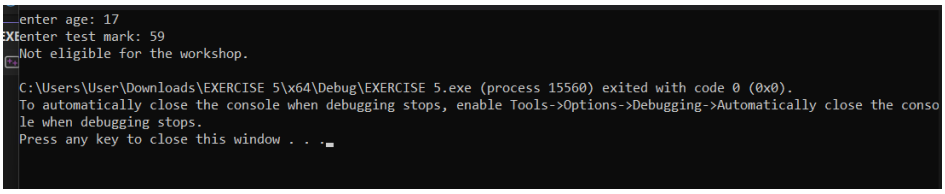
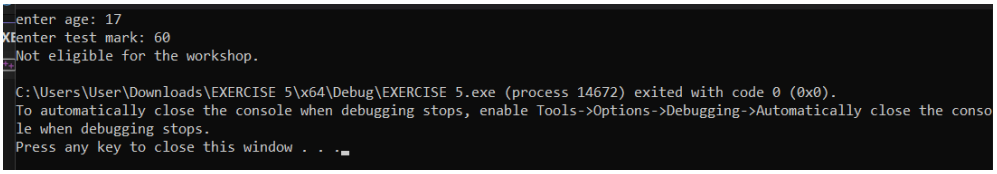
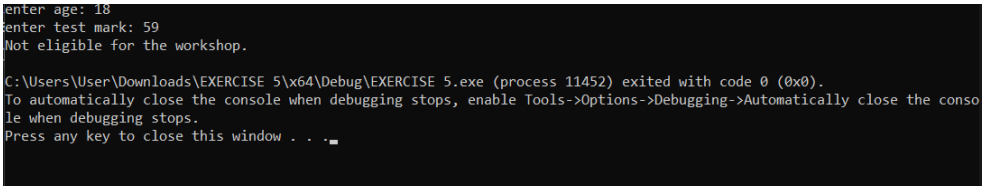
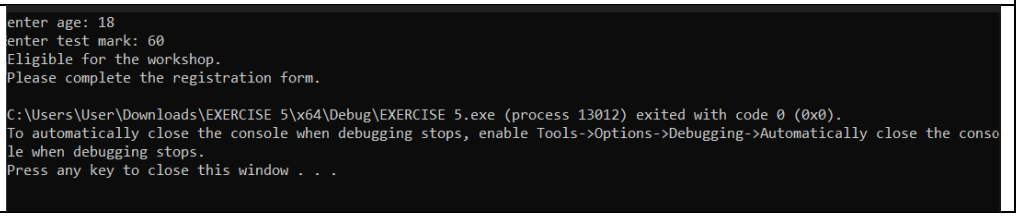
Eligible for the workshop. Please complete the registration form.



(d) Use a separate if (!eligible) statement to display Not eligible for the workshop.

```
if (!eligible)
{
    cout << "Not eligible for the workshop.\n";
}
```

(e) Test all four combinations below. Record the output and state which condition prevents eligibility in each unsuccessful case.

Age	Test mark	Expected output /reason
17	59	
17	60	
18	59	
18	60	

EXERCISE 6 : Optional: Monthly payment validator

A shop lets customers divide a purchase into equal monthly payments, without interest. Write a C++ program that reads the total purchase amount and the number of months as a double and an integer, respectively.

(a) Declare a bool variable named validPayment. Assign it a logical expression that is true only when the total is zero or greater and months is greater than zero.

```
bool validPayment = (total >= 0) && (months > 0);
```

(b) When the input is valid, calculate monthlyPayment = total / months and display the payment to two decimal places.

```
if (validPayment)
{
    double monthlyPayment = total / months;

    cout << "Monthly payment = RM" << monthlyPayment << "\n";
}
```

(c) When the input is invalid, display Invalid payment details. Use two separate if statements. The division must occur only for valid input.

Total RM	Months	Expected result
1200.00	6	enter total purchase (RM): 1200.00 enter number of months: 6 Monthly payment = RM200
1200.00	0	enter total purchase (RM): 1200.00 enter number of months: 0 Invalid payment details
1200.00	-3	Microsoft Visual Studio Debug Console enter total purchase (RM): 1200 enter number of months: -3 Invalid payment details
-100.00	6	enter total purchase (RM): -100.00 enter number of months: 6 Invalid payment details
0.00	6	enter total purchase (RM): 0.00 enter number of months: 6 Monthly payment = RM0

(d) Explain why zero months must be rejected before the calculation

Answer :

Zero months must be rejected first because division by zero is undefined and can crash the program. Checking months > 0 in validPayment makes sure the division only happens when months is valid.